Elita Lestari Setianto (201344008)	**Rangkuman Minggu 4**	Senin, 19 September 2022
		16TNK5043 (Aplikasi Mikrokontroler)
3NK		Ferry Satria, BSEE.,M.T. Rahmawati Hasanah,S.ST.,M.T.

Atmega 8535 mempunyai 4 port, dan masing masing port mempunyai 8 bit yang bisa menjadi input atau output bahkan bisa menjadi gabungan input dan output.

PORT A

PORT B

PORT C

PORT D

DDRX → Untuk mengatur bit bit pada port ABCD agar fleksibel (bisa dijadikan input atau output)

0X0F → untuk menyesuaikan port ingin dijadikan input atau ouput

0 = input

1 = output

Misalkan di port b → DDRB = 0X0F

Mengenal SREG dalam mikrokontroler AVR

Status register (SREG) adalah register berisi status yang dihasilkan pada setiap operasi yang dilakukan ketika suatu instruksi dieksekusi. SREG merupakan bagian dari inti CPU mikrokontroler. SREG merupakan bagian dari inti CPU mikrokontroler .

SREG tersusun dari 8 bit dimana masing masing bit memiliki fungsi masing-masing yaitu:

a. Bit 7

I (Global interrupt enable) bit ini digunakan untuk mengaktifkan interupsi, jika terjadi interupsi yang dipicu oleh hardware bit I akan di-clear, dan akan di-set kembali menggunakan instruksi RETI.

b. Bit 6

T (Bit Copy Storage) Instruksi BLD dan BST menggunakan bit-T sebagai sumber atau tujuan dalam operasi bit Suatu bit dalam sebuah register GPR dapat disalin ke bit T menggunakan instruksi BST, dan sebaliknya bit-T dapat disalin kembali ke suatu bit dalam register GPR menggunakan instruksi BLD.

c. Bit 5

H (Half Carry Flag)

d. Bit 4

S (Sign Bit) bit S merupakan hasil dari operasi EOR antara flag-N (negatif) dan flag-V (komplemen dua overflow).

e. Bit 3

V (Two's Complement Overflow Flag) bit ini berguna untuk mendukung operasi aritmatika.

f. Bit 2

N (Negative Flag) apabila suatu operasi menghasilkan bilangan negatif maka flag-N akan di-set.

g. Bit 1

Z (Zero Flag) Bit akan di-set bila hasil operasi yang diperoleh adalah nol.

h. Bit 0

C (Carry Flag) bit akan diset bila operasi menghasilkan carry.

1. Konversi Data desimal ke oktal cara 3 menggunakan 2 counter

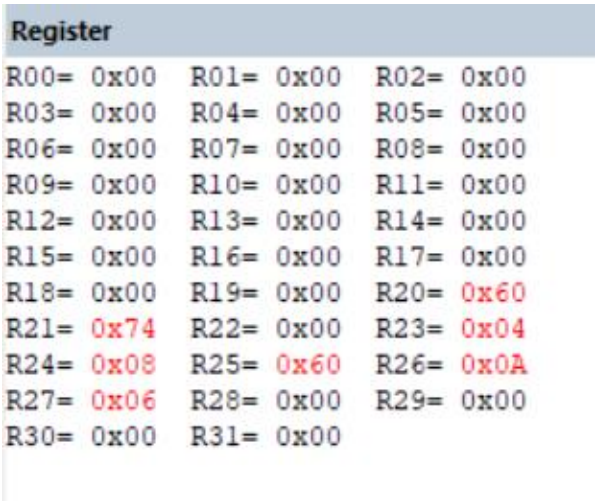
Counter 1 : menghitung turun secara desimal

Counter 2 : menghitung naik secara oktal

Program yang perlu dilakukan terlebih dahulu adalah counter 2, karena yang membatasi lupnya itu input.

Registernya nol kan dulu

```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20
;====================
ldi r20,0x60
L0:     inc r21                 ; Kesatuan Program Up Counter Oktal
        mov r23,r21
        andi r23,0x0f
        cpi r23,8
        brlt L1
        ldi r24,8               ; Secara Oktal 8 + 8 = 10
        add r21,r24
;====================
L1: INC R25
        MOV R26,R25
        ANDI R26,0X0F
        CPI R26,0X0A
        BRLT L2
        LDI R27,6
        ADD R25,R27
L2:     CP R25,R20
        BRNE L0
;====================
akhir: rjmp akhir
```



INTERUPSI ATMega 8535

Ada 3 eksternal interupsi :

INT0

INT1

INT2

- **GICR (General Interrupt Control Register)**

  Register kontrol interupsi Umum mengontrol penempatan tabel vektor interupsi.

Bit	7	6	5	4	3	2	1	0
GCIR	INT1	INT0	INT2	-	-	-	IVSEL	IVCE
Read/Write	R/W	R/W	R/W	R	R	R	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Untuk memilih/ main set interrupt mana yang akan digunakan

Untuk disable logic 0

Untuk enable logic 1

Bit 1 – IVSEL: Interrupt Vector Select

Ketika bit IVSEL dihapus (nol), vektor interupsi ditempatkan di awal memori Flash. Ketika bit ini diatur (sekali), vektor interupsi dipindahkan ke awal bagian Boot Loader Flash. Alamat sebenarnya dari awal bagian Boot Flash ditentukan oleh sekering BOOTSZ. Untuk menghindari perubahan yang tidak disengaja dari tabel vektor interupsi, Prosedur Penulisan khusus harus diikuti untuk mengubah bit IVSEL:

1. Tulis bit Pengaktifan perubahan vektor interupsi (IVCE) menjadi 1.
2. Dalam empat siklus, tulis nilai yang diinginkan ke IVSEL saat menulis 0 ke IVCE.

Interupsi secara otomatis akan dinonaktifkan saat urutan ini dijalankan. Interupsi dinonaktifkan dalam siklus IVCE diatur, dan mereka tetap dinonaktifkan sampai setelah instruksi berikut menulis ke IVSEL. Jika IVSEL tidak ditulis, interupsi tetap dinonaktifkan selama empat siklus. I-bit dalam Register Status tidak terpengaruh oleh penonaktifan otomatis.

Bit 0 – IVCE: Interrupt Vector Change Enable

Bit IVCE harus ditulis ke logika 1 untuk mengaktifkan perubahan bit IVSEL. IVCE dibersihkan oleh hardware empat siklus setelah ditulis atau ketika IVSEL ditulis.

Mengatur bit IVCE akan menonaktifkan interupsi.

- **MCUCR (MCU Control Register)**
  - ISC00 & ISC01 ➔ HUBUNGANNYA SAMA INTERRUPT 0

Table 36. Interrupt 0 Sense Control

ISC01	ISC00	Description
0	0	The low level of INT0 generates an interrupt request.
0	1	Any logical change on INT0 generates an interrupt request.
1	0	The falling edge of INT0 generates an interrupt request.
1	1	The rising edge of INT0 generates an interrupt request.

Untuk memilih dari 4 kemungkinan

- ISC11 & ISC10

Table 35. Interrupt 1 Sense Control

ISC11	ISC10	Description
0	0	The low level of INT1 generates an interrupt request.
0	1	Any logical change on INT1 generates an interrupt request.
1	0	The falling edge of INT1 generates an interrupt request.
1	1	The rising edge of INT1 generates an interrupt request.

Register kontrol MCU berisi bit kontrol untuk manajemen daya.

Bit	7	6	5	4	3	2	1	0
MCUCR	SM2	SE	SM1	SM0	ISC11	ISC10	ISC01	ISC00
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bits 7, 5, 4 – SM2..0: Sleep Mode Select Bits 2, 1, and 0

Bit ini memilih antara enam Sleep Mode yang tersedia seperti yang ditunjukkan pada Tabel di bawah:

SM2	SM1	SM0	Sleep Mode
0	0	0	Idle
0	0	1	ADC Noise Reduction
0	1	0	Power Down
0	1	1	Power Save
1	0	0	Reserved
1	0	1	Reserved
1	1	0	Standby

1	1	1	Extended Stanby

Bit 6 – SE: Sleep Enable

Bit ini harus ditulis ke logic 1 untuk membuat MCU masuk ke sleep mode saat instruksi sleep dijalankan. Untuk menghindari MCU memasuki sleep mode kecuali itu adalah tujuan pemrogram, disarankan untuk menulis bit Sleep Enable (SE) ke 1 sebelum melaksanakan instruksi sleep dan untuk menghapusnya segera setelah waking up.

- **MCUCSR (MCU Control and Status Register)**

  Kontrol MCU dan Register Status memberikan informasi tentang sumber reset mana yang menyebabkan Reset MCU.

Bit	7	6	5	4	3	2	1	0
MCUCSR	-	ISC2	-	-	WDRF	BORF	EXTRF	PORF
Read/Write	R/W	R/W	R	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	Lihat Deskripsi Bit				

Bit 3 – WDRF: Watchdog Reset Flag

Bit ini diatur jika ketika mengawasi terjadinya Reset. Bit diatur ulang dengan Reset Power-on, atau dengan menulis logika nol ke Flag.

Bit 2 – BORF: Brown-out Reset Flag

Bit ini diatur jika Reset brown-out terjadi. Bit diatur ulang dengan Reset Power-on, atau dengan menulis logika nol ke Flag.

Bit 1 – EXTRF: External Reset Flag

Bit ini diatur jika Reset eksternal terjadi. Bit diatur ulang dengan Reset Poweron, atau dengan menulis logika nol ke Flag.

Bit 0 – PORF: Power-on Reset Flag

Bit ini diatur jika Reset Power-on terjadi. Bit diatur ulang hanya dengan menulis logika nol ke Flag.

- **SREG (Status Register)**

  SREG berisi informasi tentang hasil instruksi aritmatika yang terakhir dieksekusi. Informasi ini dapat digunakan untuk mengubah aliran program untuk melakukan operasi bersyarat. Register Status diperbarui setelah semua operasi ALU, dan ditentukan dalam referensi set instruksi. SREG akan ada dalam banyak kasus, menghilangkan kebutuhan untuk menggunakan instruksi perbandingan khusus, menghasilkan kode yang lebih cepat dan lebih ringkas.

  Register Status tidak disimpan secara otomatis saat memasukkan rutinitas interupsi dan dipulihkan saat kembali dari interupsi. SREG harus ditangani oleh perangkat lunak. Register status AVR-SREG-didefinisikan sebagai:

Bit	7	6	5	4	3	2	1	0
SREG	I	T	H	S	V	N	Z	C
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	0	0	0	0	0	0	0	0

Bit 7 – I: Global Interrupt Enable

Bit GIE harus diset agar interupsi dapat diaktifkan. Kontrol pengaktifan interupsi individu kemudian dilakukan dalam register kontrol terpisah. Jika register GIE dihapus, maka tidak ada interupsi yang diaktifkan terlepas dari pengaturan pengaktifan interupsi individual. I-bit dihapus oleh perangkat keras setelah interupsi terjadi, dan diatur oleh instruksi RETI untuk mengaktifkan interupsi berikutnya. Bit I juga dapat diatur dan dihapus oleh aplikasi dengan instruksi SEI dan CLI.

Bit 6 – T: Bit Copy Storage

Bit Copy instruksi BLD (bit LoaD) dan BST (Bit STore) menggunakan T-bit sebagai sumber atau tujuan untuk bit yang dioperasikan. Sebuah bit dari register dalam file Register dapat disalin ke T dengan instruksi BST, dan bit itu dapat disalin menjadi bit dalam register dalam file Register dengan instruksi BLD.

Bit 5 – H: Half Carry Flag

Half Carry Flag H menunjukkan setengah carry membawa dalam beberapa operasi aritmetika. Half carry berguna dalam aritmetika BCD.

Bit 4 – S: Sign Bit, $S = N \oplus V$

S-bit selalu eksklusif atau antara Flag negatif N dan Flag V Overflow pelengkap keduanya.

Bit 3 – V: Two's Complement Overflow Flag

Overflow Flag V mendukung dua pelengkap operasi aritmatika.

Bit 2 – N: Negative Flag

Flag Negatif N menunjukkan hasil negatif dalam operasi aritmatika atau logika jika Flag N bernilai 1.

Bit 1 – Z: Zero Flag

Zero Flag Z menunjukkan hasil nol dalam operasi aritmatika atau logika.

Bit 0 – C: Carry Flag

Carry Flag C menunjukkan carry dalam operasi aritmatika atau logika.

- Apabila I dikasih logic 0 akan mendisable semua eksternal interrupt
- T untuk mengcopy 1 register
- Klo mau memindahkan ke T pake BST R2,3 ; simpanlah bit 3 ke r2
- Klo mau copy BLD R2,7
- H ; ada carry gak ketika penjumlahan dari bit 3 ke 4
- C; ada carry gak ketika penjumlahan bit 7 ke 8
- S, esklusif antara V or N
- V, over flow : klo logic 1 berarti terjadi over flow/under flow, klo penjumlahannya memerlukan digit lebih banyak (under flow kalau hasilnya negatif), klo logic 0 berarti; klo lebih besar dari 7 negatif
- N negative flag
  Zero : 1 (hasil aritmatikanya 0)

Program sederhana untuk menambahkan suatu nilai

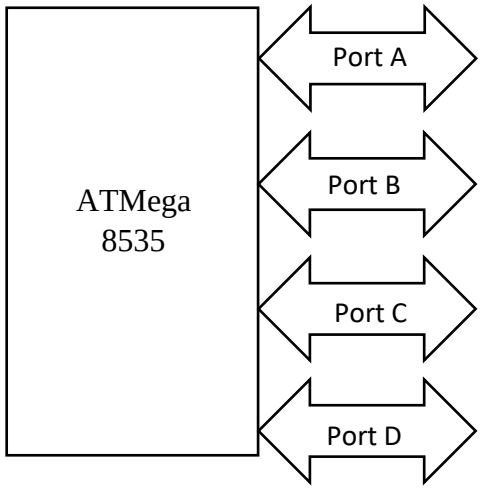
```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
main :
ldi r20,low(ramend)
out spl,r20
ldi r20,high(ramend)
out sph,r20
;====================

ldi r20,0x9c
ldi r21,0x8f
add r20,r21



akhir : rjmp akhir
```

APLIKASI ATM 8535 KE PROTEUS



Setiap Port berisi 8 Bit

Semua port dapat menjadi input maupun output

- Bila ingin menjadi output diberi logic 1; 0XFF
- Bila ingin menjadi input diberi logic 0; 0X00
- Bila ingin setengah input setengah output; 0X0F

Setiap register pada port

- A : DDRA
- B : DDRB
- C : DDRC
- D : DDRD

Contoh Program Pengaplikasian Ke Proteus

1. Membuat BCD UP Counter dan Down Counter 0 - 9

   Up Counter

```
 .include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
                ; DECIMAL UPCOUNTER, DENGAN 74LS47
                ; DELAY MENGGUNAKAN LOOP 3 REGISTER
          ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
PROTEUS_PROJECT]

MAIN:LDI R16,LOW(RAMEND)
                OUT SPL,R16
                LDI R16,HIGH(RAMEND)
                OUT SPH,R16

                LDI R20,0XFF
                OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:    LDI R18,0 ;buat counter
SANA:           OUT PORTB,R18 ;port b sebagai output
```

```
                RCALL DELAY
                INC R18 ;naikin
                CPI R18,0X0A ;
                BRNE SANA
                RJMP ITU

DELAY:      LDI R23,3 ;buat ngatur delay
LUP1: LDI R24,0XFF ; buat output
LUP2: LDI R25,0 ; masukin r25 biar jadi nol
LUP3: DEC R25 ; naik
                BRNE LUP3
                DEC R24
                BRNE LUP2
                DEC R23
                BRNE LUP1
                RET
```

Down Counter

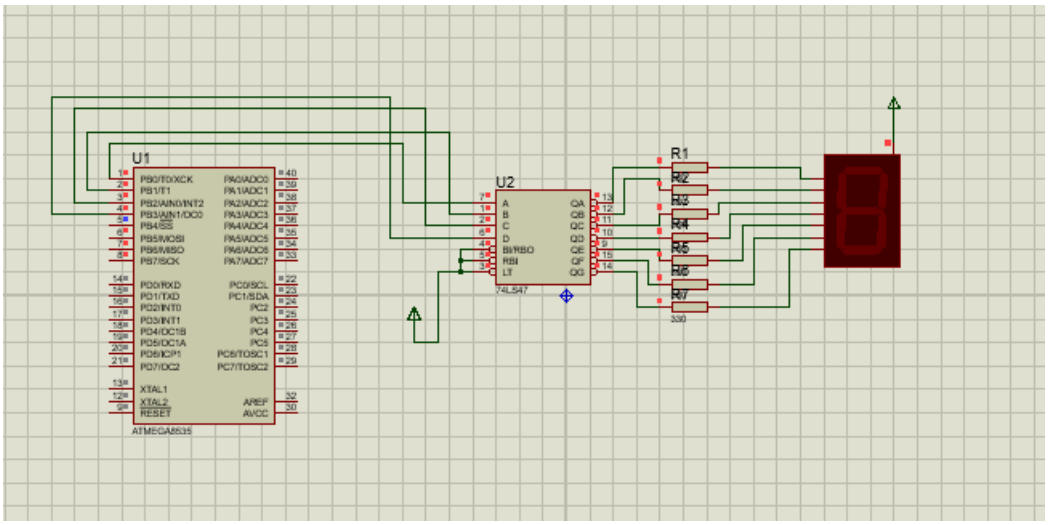
```
.include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
                ; DECIMAL UPCOUNTER, DENGAN 74LS47
                ; DELAY MENGGUNAKAN LOOP 3 REGISTER
          ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
PROTEUS_PROJECT]

MAIN:LDI R16,LOW(RAMEND)
                OUT SPL,R16
                LDI R16,HIGH(RAMEND)
                OUT SPH,R16

                LDI R20,0XFF
```

```
                OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:    LDI R18,9 ;buat counter
SANA:           OUT PORTB,R18 ;port b sebagai output
                RCALL DELAY
                DEC R18 ;naikin
                CPI R18,0 ;
                BRNE SANA
                RJMP ITU

DELAY:          LDI R23,3 ;buat ngatur delay
LUP1: LDI R24,0XFF ; buat output
LUP2: LDI R25,0 ; masukin r25 biar jadi nol
LUP3: DEC R25 ; naik
                BRNE LUP3
                DEC R24
                BRNE LUP2
                DEC R23
                BRNE LUP1
                RET
```



2. Membuat BCD UP Counter Modulo 26 (0-25). Gunakan 7-segemnet Common Anode

```
 .include "m8535def.inc"
.org 0x00
rjmp main
        ; PAKAI PROJECT "ROOT"
                ; DECIMAL UPCOUNTER, DENGAN 74LS47
                ; DELAY MENGGUNAKAN LOOP 3 REGISTER
          ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
PROTEUS_PROJECT]

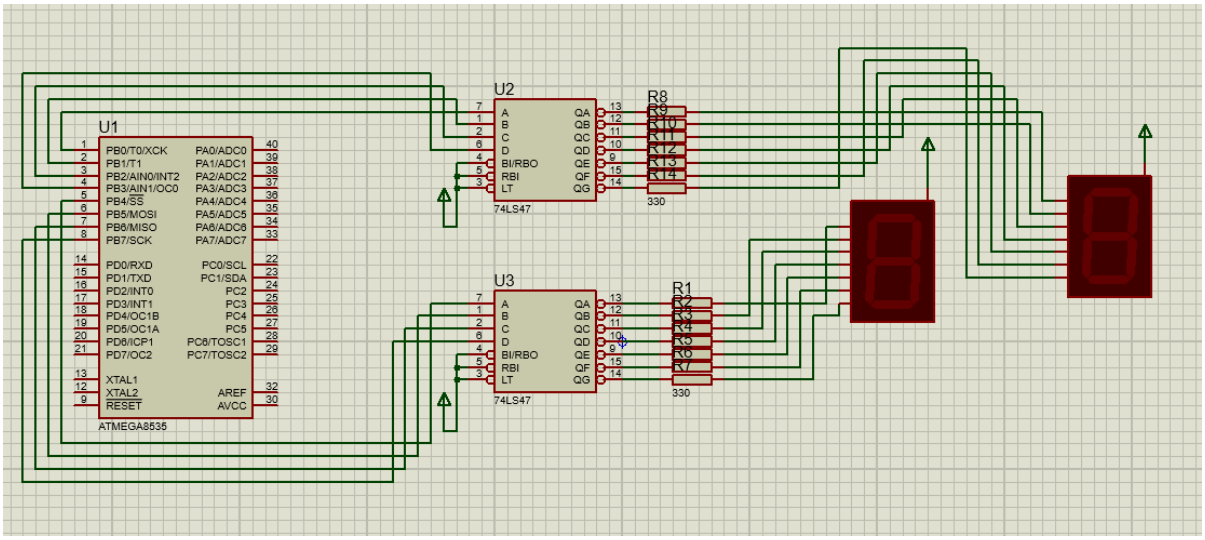
MAIN:LDI R16,LOW(RAMEND)
                OUT SPL,R16
                LDI R16,HIGH(RAMEND)
                OUT SPH,R16

                LDI R20,0XFF
                OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:    LDI R18,0 ;buat counter
SANA:           OUT PORTB,R18 ;port b sebagai output
                RCALL DELAY
                LDI R20,1
                ADD R19,R20
                RCALL DAA
                MOV R18,R19
                CPI R18,26
                BRNE SANA
                RJMP ITU

DELAY:          LDI R23,3 ;buat ngatur delay
LUP1: LDI R24,0XFF ; buat output
LUP2: LDI R25,0 ; masukin r25 biar jadi nol
LUP3: DEC R25 ; naik
                BRNE LUP3
                DEC R24
```

```
                BRNE LUP2
                DEC R23
                BRNE LUP1
                RET
daa:    push r16;amankan isi reg. ke stack
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
        ;       rjmp lup5
lup4:   mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
                rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
        ;       rjmp lup7
lup6:   mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
```

```
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:
                out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
tuh:    out sreg,r16
                adc r24,r24
                ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
                out sreg,r16
                add r19,r21
                in r16,sreg
                adc r24,r24; amankan carry pd r24
                ret ; akhir dari prog. subrutin.
```



3. Membuat BCD Down Counter Modulo 31 (31-0). Gunakan 7-segemnet Common Anode

```
 .INCLUDE"M8535DEF.INC"
.ORG 0
RJMP MAIN
                ; BCD DOWN COUNTER MODULO 30, DENGAN 74LS47
                ; DELAY MENGGUNAKAN LOOP 3 REGISTER
            ; IMPLEMENTASI: PROTEUS DEC_COUNT_01 [DI
PROTEUS_PROJECT]

MAIN:LDI R16,LOW(RAMEND)
                OUT SPL,R16
                LDI R16,HIGH(RAMEND)
                OUT SPH,R16

                LDI R20,0XFF
                OUT DDRB,R20 ; PORT B SBG OUTPUT
ITU:    LDI R18,0X30
SANA:           OUT PORTB,R18
                RCALL DELAY
                LDI R20,0X99
                ADD R18,R20
                MOV R19,R18
```

```
                RCALL DAA
                MOV R18,R19
                CPI R18,0X99
                BRNE SANA
                RJMP ITU

DELAY:      LDI R23,3
LUP1: LDI R24,0XFF
LUP2: LDI R25,0
LUP3: DEC R25
                BRNE LUP3
                DEC R24
                BRNE LUP2
                DEC R23
                BRNE LUP1
                RET
daa:    push r16;amankan isi reg. ke stack
                push r17
                push r21
                push r24
                in r16,sreg; amankan sreg
                in r17,sreg
                ldi r24,0 ; r24 hanya sbg status dr carry flag
                adc r24,r24; status carry flag sebelum koreksi
; ambil, amankan kembali sreg, uji half carry
                out sreg,r17
                brhc lup4; lompat ke lup4 jika H=0
                rcall fk_ln; satuan + 06 jika H=1
        ;       rjmp lup5
lup4:   mov r21,r19
                andi r21,$0f; ambil dgt.satuan
                cpi r21,$0a ; > 9
                brcs lup5; lompat ke lup5 jika < 9
```

```
                rcall fk_ln; satuan + 06 jika > 9
lup5:   out sreg,r17; ambil, amankan kembali sreg, uji carry
                brcc lup6; lompat ke lup6 jika C=0
                rcall fk_hn; puluhan + 60 jika C=1
        ;       rjmp lup7
lup6:   mov r21,r19
                andi r21,0xf0; ambil dgt.puluhan
                cpi r21,0xa0
                brcs lup7; lompat ke lup7 jika pul < a
                rcall fk_hn; pul + 60 jika > 9
lup7:   out sreg,r16; mengatur carry flag sblm keluar:
                clc; C = 0
                in r16,sreg ;amankan sreg stlh c=0
                cpi r24,0 ;menguji status
                breq lup8
                out sreg,r16;ambil sreg, set c
                sec; C = 1
                in r16,sreg; amankan sreg stlh c=1
lup8:
                out sreg,r16;status akhir
                pop r24;kembalikan isi reg. dr stack
                pop r21
                pop r17
                pop r16
                ret
fk_ln:  ldi r21,06; koreksi dgt.satuan
                out sreg,r16
                add r19,r21
                in r16,sreg
                brcc tuh
                ldi r21,0x01
                add r17,r21
tuh:    out sreg,r16
```

```
                adc r24,r24
                ret
fk_hn:  ldi r21,$60; koreksi dgt.puluhan
                out sreg,r16
                add r19,r21
                in r16,sreg
                adc r24,r24; amankan carry pd r24
ret ; akhir dari prog. subrutin.
```

